

Data Engineering Architecture

How data flows from where it's created to dashboards and AI — every component explained

The running example — "ShopFast," an online store. Follow one goal through the whole system: "Show the sales team a dashboard of yesterday's revenue by region, and feed the recommendation model." Each component below shows what it does and its part in this story.

The architecture at a glance



RUNS ACROSS EVERY LAYER (THE SUPPORT SYSTEMS)

Orchestration — schedules & sequences the pipeline with retries. Airflow, Step Functions.

Governance & Security — access control, encryption, PII. IAM, KMS, Lake Formation.

Data Quality — tests that stop bad data before users see it. dbt, Great Expectations.

Monitoring & Cost — alarms on failures, latency, spend. CloudWatch, Spark UI.

Infra as Code & CI/CD — platform defined as code, deployed safely. Terraform, Git.

The Data Engineer — designs, builds, and runs all of it so clean data ships daily.

Each component — explained with the ShopFast story

1. Sources — where data is born

Every action a company records: an order placed, a button clicked, a payment made, a sensor reading.

Data engineer's job: connect to these systems and pull data out reliably without slowing them down.

SHOPFAST EXAMPLE

A customer buys sneakers. The order is saved in ShopFast's **orders database**; the pages they browsed are recorded as **click events**. That's the raw data being created.

2. Ingestion — bringing the data in

Some data arrives on a schedule (a nightly copy = **batch**); some arrives continuously, event by event (a live feed = **streaming**).

Data engineer's job: set up the pipes, handle retries, and make sure nothing is lost or duplicated.

SHOPFAST EXAMPLE

New and changed orders are copied from the database every few minutes using **DMS (CDC)**. Clicks pour in all day through **Kinesis** and are dropped into storage as they happen.

3. Data Lake — the big cheap storage room

All data lands here first, exactly as it arrived, in very cheap storage (Amazon S3), organized into zones: **raw** (untouched), **clean** (fixed up), **curated** (ready to use).

Data engineer's job: design the zones and folder layout, use Parquet, and partition by date so reads are fast and cheap.

SHOPFAST EXAMPLE

Yesterday's orders sit at **s3://shopfast/raw/orders/dt=2026-08-10/** — untouched. Nobody edits raw; if a mistake is found later, we can always reprocess from it.

4. Processing with Spark — the heavy lifting

Messy data becomes useful data: cleaning, joining tables, adding things up. Because there's too much for one machine, **Spark** splits the work across many machines running at the same time.

Data engineer's job: write the transformations (in PySpark) and tune them so big jobs run fast and cheap.

SHOPFAST EXAMPLE

A **Spark job** reads the **2 TB** of orders, drops cancelled ones, joins in region names, and adds up **revenue per region**. 40 workers chew through it in parallel and write the result to the **curated** zone.

5. Catalog — the labels on the shelves

After processing, we record what each table is and its columns, in one central place, so any query tool can find and understand the data.

Data engineer's job: keep the catalog current (crawlers or definitions) so analysts can just write SQL.

SHOPFAST EXAMPLE

A **Glue Crawler** registers the new **revenue_by_region** table (columns: region, revenue). Now Athena and Redshift can see it — no one has to know the file paths.

6. Warehouse / Serving — where questions get answered

The clean, organized data goes somewhere built for fast questions — a **data warehouse** (Redshift/Snowflake) — or is queried directly on the lake with Athena.

Data engineer's job: model the data into easy tables and load it efficiently so dashboards respond in seconds.

SHOPFAST EXAMPLE

The table is loaded into **Snowflake**. An analyst runs **SELECT region, revenue FROM revenue_by_region ORDER BY revenue DESC** and gets the answer instantly.

7. Consumers — the people who use it

Finally, the data powers **dashboards** (charts business teams watch), **AI/ML models**, and apps.

Data engineer's job: make sure the data behind them is correct, fresh, and trustworthy.

SHOPFAST EXAMPLE

The sales team opens a **QuickSight dashboard** and sees **Texas led yesterday's revenue**. Separately, the **recommendation model** uses the click features to suggest products. Goal achieved.

The full story in one line: a customer's order and clicks (sources) are pulled in (ingestion), stored cheaply (data lake), cleaned and summed into "revenue by region" by Spark (processing), labeled (catalog), loaded into Snowflake for fast SQL (serving), and shown on the sales dashboard and fed to the reco model (consumers) — scheduled, secured, quality-checked, and monitored the whole way.